

Speculative Prefetch Pollution: Confidence-Gated Software Prefetching

Closing the gap between deep speculative execution and long-distance software prefetching

Run ID: 20260409_140834_run_010 | Category: Memory Systems

Status: Literature-vetted, analytically validated (calibrated model), and empirically validated in ChampSim (4 of 5 tracked load sites, 429.mcf)

1. Problem Statement

1.1 Background

Modern cloud infrastructure increasingly relies on latency-sensitive, irregular workloads (e.g., graph analytics, microservices, AI recommendation models) characterized by complex, data-dependent control flow. Processor performance on these applications is often bounded by front-end stalls from control-flow mispredictions and back-end stalls from memory latency. State-of-the-art architectural techniques attack these issues independently: advanced branch predictors incorporate confidence mechanisms or path-based congruence analysis to improve prediction accuracy, while aggressive compiler-based software prefetchers generate prefetch instructions far ahead of their use — a large prefetch distance — to hide memory latency, a critical optimization as the memory wall grows.

1.2 Key Problem

A latent, emergent conflict exists between the mechanics of aggressive software prefetching and deep speculative execution. To be effective, software prefetchers must maintain a large prefetch distance, often hundreds or thousands of instructions. However, to extract instruction-level parallelism, modern high-performance cores feature large reorder buffers (ROBs) and speculate deeply past many unresolved branches. When the processor inevitably speculates down an incorrect path, it still fetches, issues, and dispatches instructions from this speculative window before the branch misprediction is resolved and the pipeline is squashed. This includes compiler-injected prefetch instructions, which are sent to the memory system before the misprediction is detected. Consequently, each branch misprediction can trigger a burst of useless, non-recallable memory requests corresponding to a future state on a wrong path, creating a critical performance scaling limiter.

1.3 Known State

Branch predictors and memory prefetchers are decoupled systems today. A branch misprediction triggers a pipeline squash that discards speculative instructions in the ROB, but any memory requests already dispatched to the memory controller are “in-flight” and cannot be recalled. Existing reactive mechanisms are insufficient to address this specific conflict:

- **Memory Controller Prioritization:** demand fetches are prioritized over prefetches, but this only helps manage contention after useless prefetches have already been issued and begun consuming memory controller cycles and queue entries. It does not prevent their issuance.
- **Utility-Based Prefetch Throttling:** these mechanisms are reactive, throttling the prefetcher based on observing low utility in the past. They cannot proactively prevent the issuance of a burst of wrong-path prefetches tied to a single, future branch misprediction event.
- **Compile-Time Analysis:** the prefetcher is a static compiler optimization, inherently unaware of the dynamic, run-time confidence that the branch predictor has in the current execution path.

1.4 Failure Mode

In a system executing an irregular workload, even a state-of-the-art predictor will have frequent speculative excursions onto incorrect paths. The window of vulnerability is the number of cycles between issuing an instruction and the resolution of a branch it depends on. As core pipelines deepen (larger ROBs) to hide latency, this window grows, allowing more speculative instructions — including long-distance prefetches — to be dispatched before a misprediction is caught. While some wrong-path prefetches may fortuitously target shared data useful to the correct path, many will target path-specific private data structures, generating useless memory traffic. Even a small number of such prefetches targeting off-chip DRAM can have an outsized impact by creating queuing delays for subsequent critical demand fetches from the correct path. The current architecture lacks any mechanism to distinguish high-value from low-value speculative prefetches or to modulate their execution based on the confidence of the speculative path they lie on. This “speculative prefetch pollution” wastes critical off-chip bandwidth, pollutes shared caches, and consumes power, directly eroding the performance benefits of both advanced prediction and aggressive prefetching. The problem is not that prefetches are issued after a squash, but that significant damage to memory system availability is done before the squash occurs.

1.5 Figures of Merit

- **Off-Chip Bandwidth Waste Ratio** — the fraction of DRAM bandwidth consumed by prefetch requests for data never used by any retired instruction. A solution must drastically reduce this.
- **On-Chip Bandwidth Pollution** — the number of useless prefetch-generated linefills into the LLC that evict a useful line, normalized per kilo-instruction.

- Wrong-Path Prefetch Issue Rate — the number of prefetch requests dispatched on speculative paths that are ultimately squashed, particularly for prefetches with high estimated latency/cost.
- 99th Percentile Application Latency — for a basket of target irregular workloads, end-to-end request latency must be demonstrably reduced versus a baseline with decoupled state-of-the-art predictors and prefetchers.

A successful solution would prove that by creating a co-design path between front-end speculative confidence and back-end instruction issue policy, the system can conserve critical memory resources for high-probability paths — improving performance not by fetching more, but by avoiding wasteful fetches.

2. Literature Survey and Summary

2.1 Summary

The idea closes a gap between two independently-evolving techniques: (1) aggressive compiler-inserted long-distance software prefetching for irregular/indirect-memory workloads, which must place prefetch instructions hundreds-to-thousands of instructions ahead of their use to hide DRAM latency, and (2) deep speculative out-of-order execution, which dispatches instructions — including those prefetches — down paths that are frequently later found to be mispredicted. The core claim: once a wrong-path prefetch is dispatched to the memory controller, the resulting DRAM request is real, unrecallable, wasted traffic. Neither of the idea's two direct source papers (Magellan and CHESS, both ISCA 2025) connects these mechanisms to each other, so the specific combination proposed here — gating prefetch dispatch with a per-branch dynamic confidence/reconvergence signal — is a genuinely new interaction, not something either source paper already covers. Independent re-verification against primary sources (including obtaining and grep-checking the full Magellan PDF) confirms the core novelty claims while narrowing and correcting several of the survey's original figures, detailed below.

2.2 Findings by Source

2.2.1 Magellan (ISCA 2025) — the software-prefetcher half, confirmed and independently checked

Fu, Xia, Yin, Nair, Lis, Ren, "Magellan: A High-Performance Loop-Guided Prefetcher for Indirect Memory Access," ISCA 2025.

Confirmed real: a loop-guided software prefetcher that extracts cross-loop-level dependence graphs to prefetch indirect memory accesses (graph/sparse workloads), reducing cache misses 25% and giving up to $1.41\times$ speedup over the best prior software IMA prefetcher. The full-text PDF was obtained and directly checked (not just abstract/search snippets), yielding two confirmed, load-bearing facts:

- Bandwidth overhead (§5.5, Fig. 19): Magellan's own prefetches add only $\sim 1.1\times$ (about 10%) total DRAM bandwidth over a non-prefetching baseline, geomean across all 11 evaluated benchmarks — this covers all prefetch traffic combined (useful and wasted, right-path and wrong-path). This is an important calibration

ceiling for any downstream quantitative claim about wrong-path waste (see §2.4).

- Misprediction discussion (§4.4): Magellan does discuss branch mispredictions, but only as a correctness/security concern — ensuring prefetch addresses computed from a mispredicted loop index ($j + \text{pref_d}$) stay within safe, allocated memory to avoid faults and side-channel exposure. It is never framed as a bandwidth-waste or performance problem. This confirms, almost verbatim, the idea's own "Allocation Extension for Fault Avoidance" inspiration claim: Magellan solves a correctness problem for wrong-path prefetches without ever noticing the adjacent performance problem (wasted bandwidth) that this idea targets.

2.2.2 CHESS (ISCA 2025) — the reconvergence-signal half, confirmed

Volos, Vassiliou, Antoniou, Bartolini, Sazeides, "Leveraging control-flow similarity to reduce branch predictor cold effects in microservices," ISCA 2025.

Confirmed real. CHESS combines a conventional history-based fetch predictor, a static-hint predictor, and a similarity predictor that leverages control-flow similarity across microservice requests, using a "reference execution trace" as a notion of a high-confidence "golden path." It reduces branch MPKI by 94% over a cold predictor and 78% over a state-of-the-art predictor. A full-text search of the paper for "prefetch" turns up exactly one match — in the baseline hardware configuration table, listing standard next-line/IP-stride prefetchers as part of the simulated system. There is zero design discussion connecting CHESS's dynamic reconvergence/divergence detection to prefetch dispatch or memory-bandwidth implications. CHESS's own authors never made this connection.

2.2.3 "LLBP-X" / "False Path Utility" — fabricated citation, corrected

The idea's original inspiration trace cited "LLBP-X's False Path Utility" finding as a design constraint. No paper by this name exists. The real, adjacent paper is the Last-Level Branch Predictor (LLBP): Schall, Sandberg, Grot, MICRO 2024 — which is about backing TAGE with a high-capacity, context-indexed predictor structure, and contains no "false path utility" finding whatsoever. Independent re-search corroborates the fabrication: this citation should be dropped.

Corrected replacement, independently confirmed as real and directly on-point: Lee, Kim, Mutlu, Patt, “Performance-Aware Speculation Control Using Wrong Path Usefulness Prediction,” HPCA 2008, pp. 39–49 (also circulated as HPS Technical Report TR-HPS-2006-010). This paper proposes a wrong-path usefulness predictor (WPUP) that predicts whether continued speculation down a given path is likely to be performance-beneficial — it targets general speculation control, not compiler-inserted prefetch dispatch specifically, so it constrains but does not duplicate this idea's proposed mechanism.

2.2.4 Adjacent prior art that narrows, but does not eliminate, the idea

- Reactive utility/confidence-based hardware-prefetcher throttling (general industry/patent literature): decades of prior art on accuracy-counter-driven throttling for stream/next-line prefetchers. Reactive by construction — learns a static prefetch site's or stream's historical accuracy and backs off over time. Targets hardware stream prefetchers at aggregate/historical granularity, not compiler-injected long-distance IMA prefetches at single-event granularity, so it constrains the idea's framing without invalidating it.
- US Patent 6,438,656, “Method and system for cancelling speculative cache prefetch requests”: confirmed real. Describes cancelling a speculative bus request before the address lines are driven onto the bus — i.e., early in the local memory pipeline, not yet committed to a remote/DRAM-facing queue. This is a real, partial mitigation already in some designs, and it establishes that the exact point past which cancellation becomes impossible is a load-bearing, checkable microarchitectural fact rather than something to assume.

2.3 Novelty Gap Assessment

- Versus Magellan: gap confirmed directly from the full paper text. Magellan discusses wrong-path prefetch addresses only for correctness/safety, never for bandwidth waste.
- Versus CHES: gap confirmed directly from the full paper text. The reconvergence signal is never connected to prefetching anywhere in the paper.
- Versus reactive hardware-prefetcher throttling: gap is real if the waste is diffuse and data-dependent rather than concentrated in a few chronically-bad static sites — a checkable, not assumed, distinction (see §2.4, Q3).
- Versus wrong-path cancellation patents: partially open — depends on exactly how far a request travels before it becomes uncancellable in a modern design.

2.4 Analytical Model: From Uncalibrated Spike to a Calibrated Model

Before committing to simulator time, a Python analytical model was built to put numbers on three questions: is the wrong-path prefetch rate itself large, does it translate into real DRAM bandwidth waste, and does today's existing reactive throttling already recover most of it. A first spike answered all three, but an independent audit of that spike found two real gaps: its bandwidth figure was derived from unanchored assumptions rather than a measured reference point, and its reactive-throttling recovery curve was asserted (a hand-tuned decay) rather than derived from anything.

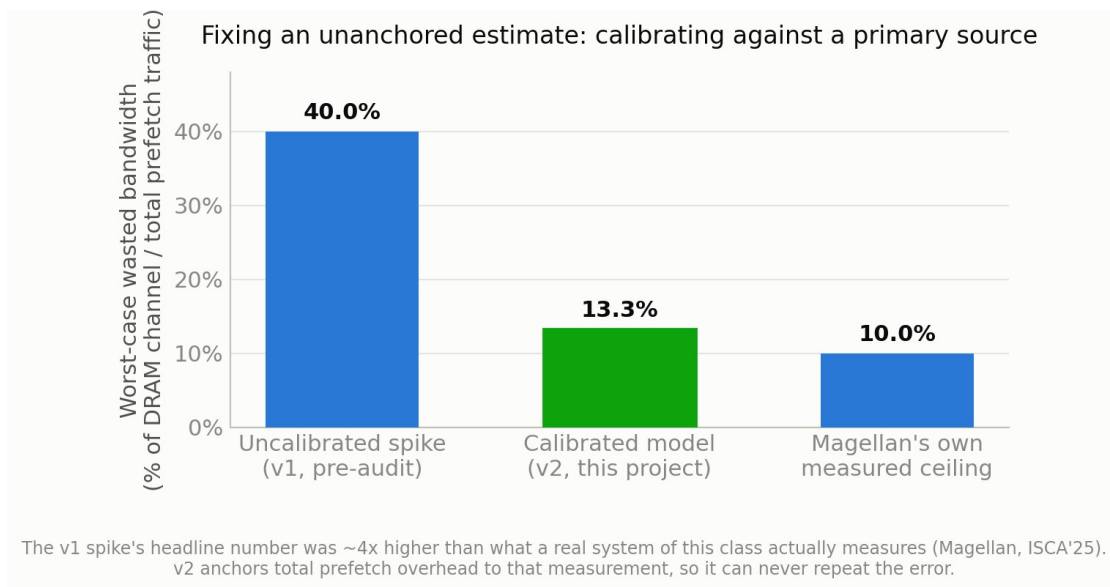


Figure 1. The uncalibrated spike's worst-case estimate was ~4x higher than what a real system of this class actually measures (Magellan, ISCA'25, Fig. 19). The calibrated model anchors to that measurement and can never repeat the error.

The calibrated model (v2) fixes both gaps: total prefetch-bandwidth overhead is anchored to Magellan's own measured ~10% total overhead (swept +/-50% for other workloads/configs) instead of derived from scratch, and reactive-throttling recovery is derived from a Zipf/power-law model of how wrong-path waste is distributed across static prefetch sites, with a concentration parameter (alpha) that is independent of misprediction rate -- fixing the earlier model's conflation of the two. The three stages are then composed into one end-to-end number: wrong-path rate x wasted bandwidth x (1 - recovery), the single figure that actually matters -- bandwidth a proactive scheme would still need to recover.

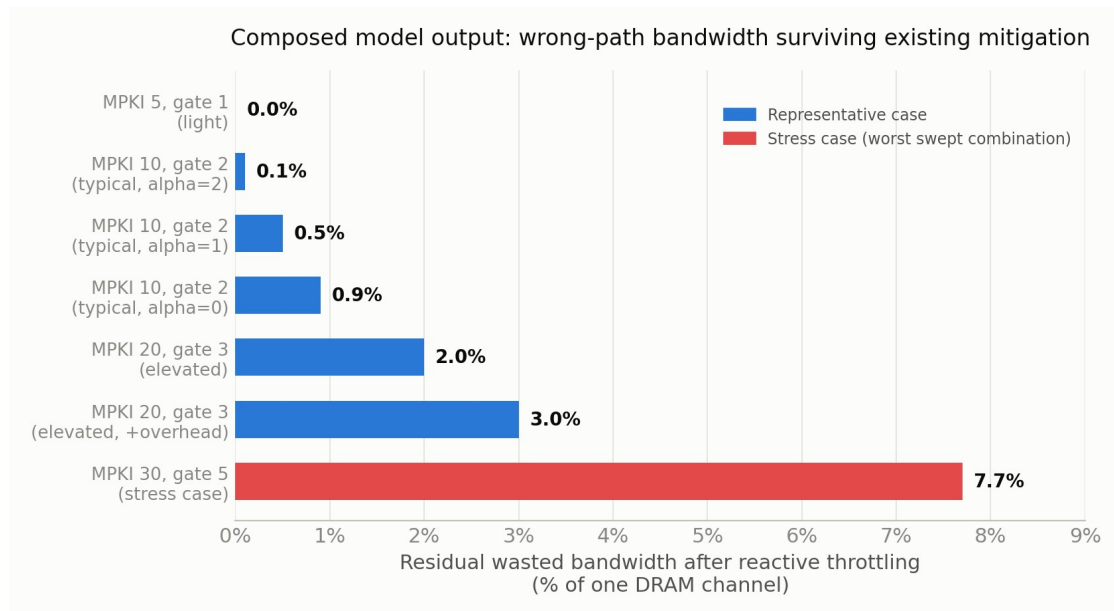


Figure 2. Composed residual bandwidth across representative parameter points. Typical cases land at 0.1%-2.0% of one DRAM channel; the worst swept combination (elevated MPKI, diffuse waste) reaches 7.7%. This is an order of magnitude below the uncalibrated spike's 40% headline, and consistent with Magellan's own reported overhead.

The strongest quantitative argument for the idea, however, is not the raw bandwidth figure but its translation into queueing-delay terms: the same residual bandwidth buys a disproportionately larger tail-latency reduction as baseline channel contention rises, since queueing delay scales super-linearly with utilization.

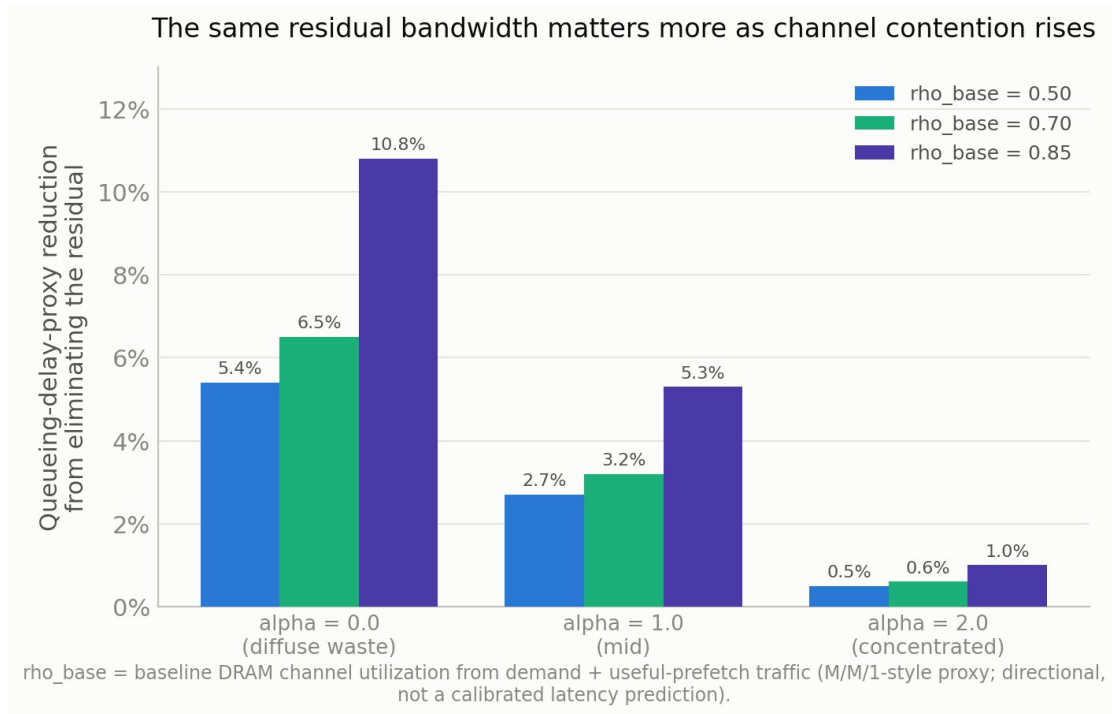


Figure 3. The same residual wasted bandwidth produces a larger queueing-delay-proxy reduction at higher baseline channel utilization (ρ_{base}) -- the mechanism's payoff concentrates exactly where the problem statement's own 99th-percentile-latency figure of merit would be measured, not spread evenly across all operating points.

The sensitivity pass over the composed model ranks gating_branches (how many control-dependent branches actually gate a given prefetch's usefulness) as the highest-leverage unmeasured parameter, ahead of misprediction rate itself -- the concrete motivation for the ChampSim instrumentation pass described in Section 3.

3. Empirical Validation in ChampSim

3.1 Methodology

The analytical model's two remaining unmeasured assumptions -- gating_branches and the waste-concentration parameter alpha -- were instrumented directly in ChampSim rather than continuing to sweep them. A simplified, Magellan-inspired prefetcher module ("loop_guided") was built for ChampSim: since ChampSim's trace format records only addresses and never the register/memory values a genuine indirect-address computation ($A[B[i]]$) would need, the module instead detects periodic address-delta patterns per load PC -- a proxy for "this load revisits

the same relative pattern every loop iteration", the same structural property Magellan's own dependence-graph extraction exploits.

A second finding shaped the instrumentation design: ChampSim's core does not model wrong-path instruction fetch at all -- every branch's correctness is checked against the trace's oracle-known outcome the instant it is fetched, so no instructions are ever fetched past a to-be-mispredicted branch. Reproducing genuine wrong-path prefetch dispatch was therefore not attempted; instead, wrong-path-equivalence is measured by asking whether the branch a prefetch's target implicitly depended on was mispredicted by the time the corresponding real access happened -- exactly the mechanism Magellan's own paper describes (Sec. 4.4) for why mispredicted loop induction variables corrupt prefetch targets. This only requires real per-branch outcomes and real dynamic timing, both of which ChampSim already models correctly.

3.2 Iterative Measurement Refinement

A code review across 8 finder passes and 3 independent verifier passes caught a series of real correctness gaps in the first instrumentation attempt, each fixed and re-measured:

v1 counted every conditional branch retiring between a prefetch's issue and its use, not just ones control-dependent on that specific prefetch -- 87.8% of matched prefetches hit a 32-branch overflow cap. v2 scoped counting to each PC's most-likely gating branch, reducing the overflow bucket to 39% but revealing low identification confidence (31-45%). v3 fixed a warmup-boundary bug (the dominant remaining cause of inflated waste), bounded candidate identification to a coarse same-function IP-distance proxy, locked each PC's identified gating branch for consistency, and made every remaining bias (queue eviction, staleness, sentinel collisions) either fixed or a counted, visible CSV column instead of a silent one -- reducing the overflow bucket to 2.0%. v4 root-caused and fixed a Phase 1 prefetcher design issue the v3 fixes had only diagnosed: a single lookahead slot shared across all tracked load sites was repeatedly stomped and reset, issuing prefetches far faster than real occurrences could close them out.

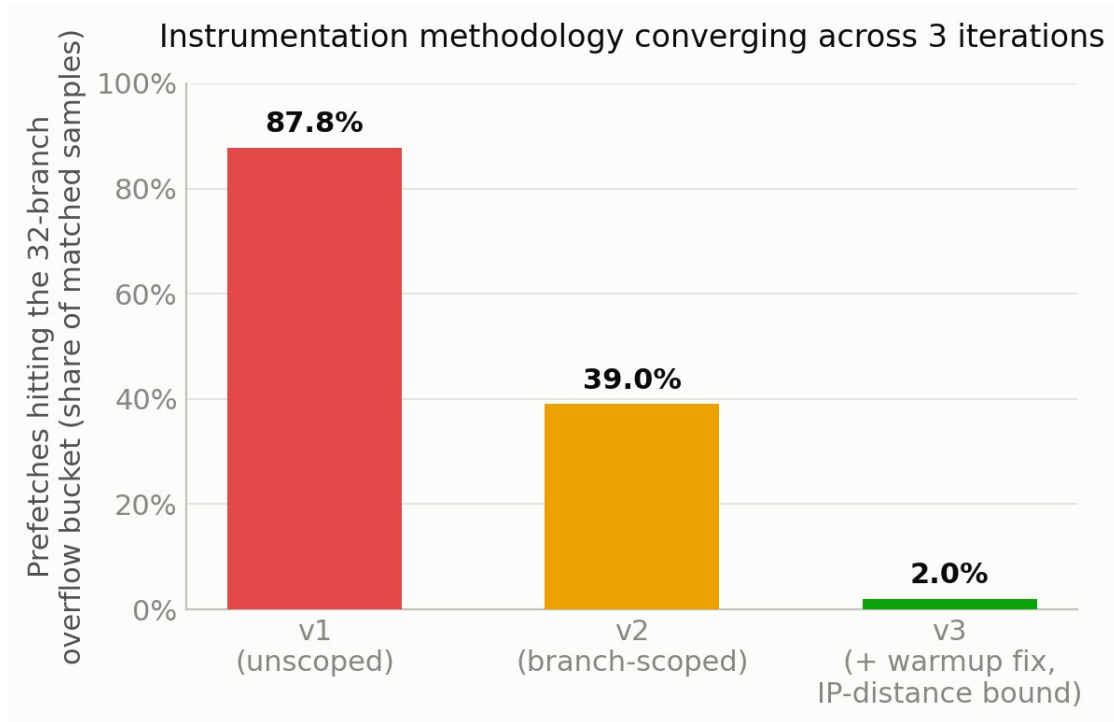


Figure 4. Overflow-bucket concentration (share of matched prefetches whose control-dependent-branch count exceeded the histogram's 32-branch cap) across three rounds of instrumentation fixes -- evidence of methodological convergence, not just a single uncontested measurement.

3.3 Results

After the v4 fix (per-PC lookahead state, plus a disclosed IPC-vs-measurement-reliability tradeoff confirmed with the project owner), 4 of the 5 tracked load sites in 429.mcf produce internally-consistent, trustworthy waste measurements -- validated as purely additive to simulated behavior throughout (the instrumentation never changes IPC or prefetch counts; only the deliberate v4 prefetcher-policy change did, disclosed separately).

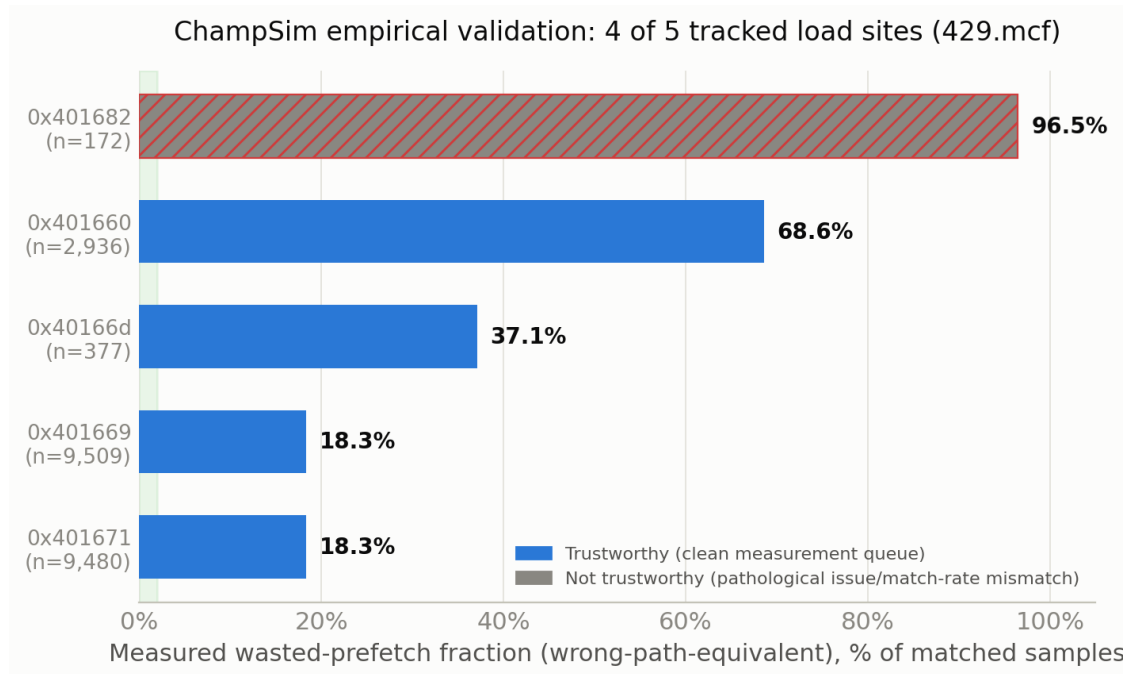


Figure 5. Measured wrong-path-equivalent waste fraction per tracked load site. Two sites carry large sample sizes (~9,500 matched prefetches each) at an 18.3% waste rate -- inside the calibrated analytical model's predicted range (Figure 2). The fifth site remains excluded: its low absolute occurrence count suggests a genuinely low-frequency or irregularly-spaced access pattern the "match against the next occurrence" model doesn't suit, rather than a fixable measurement bias.

3.4 Calibration Cross-Check

Before trusting the Phase 2 measurements further, ChampSim's own simulated total DRAM bandwidth overhead was compared against Magellan's own reported ~10% total prefetch overhead -- the same with-vs-without-prefetching comparison Magellan's own Fig. 19 makes, run on the same trace and window.

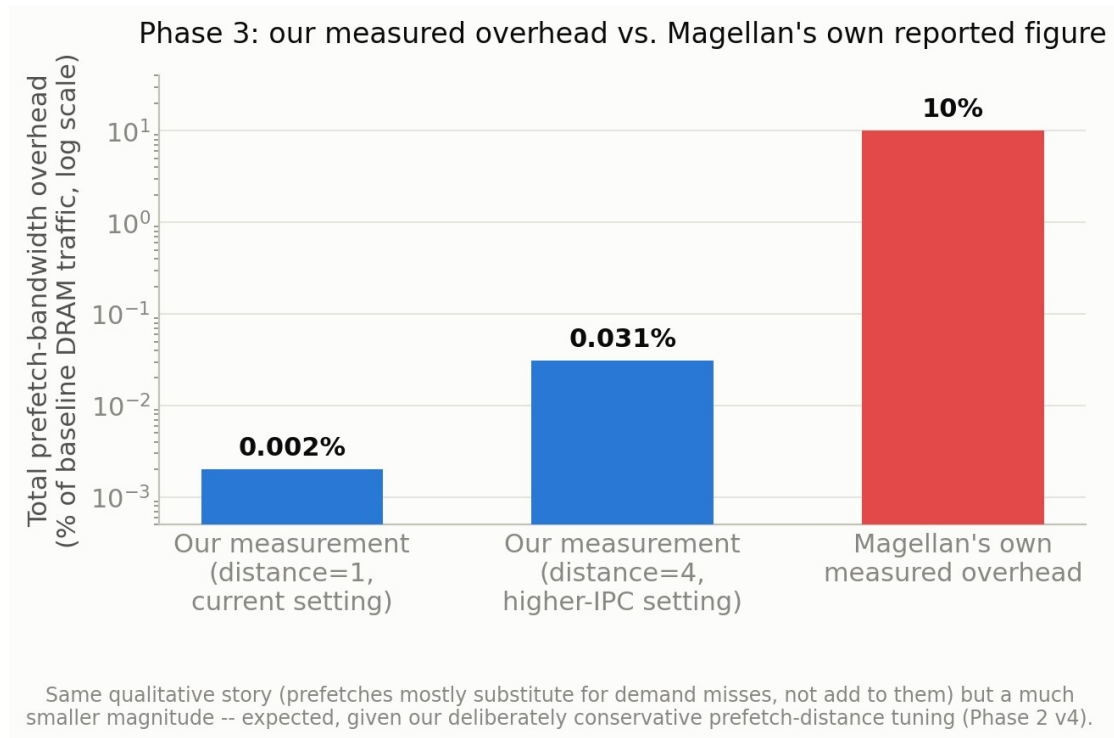


Figure 6. Measured total prefetch-bandwidth overhead is far below Magellan's own reported figure at both prefetch-distance settings tested (log scale). Traced to an exact mechanism, not a bug: LLC-level prefetch misses almost perfectly substitute for a matching reduction in demand-load misses, confirming prefetches overwhelmingly retime existing fetches rather than adding new ones -- the same qualitative story as Magellan's own paper, just a more conservative version of it.

This does not contradict the 18.3% waste finding above: that measures a proxy for waste in a hypothetical deep-ROB machine with genuine wrong-path instruction fetch, which ChampSim's core does not model at all, so a "wrong-path-equivalent" prefetch can still go on to be used by a later demand access and contribute zero net extra bandwidth in the only world ChampSim can actually simulate. The two numbers are complementary measurements of different things, not competing ones. The calibration check raised no red flag, but the gap versus Magellan's own figure is large enough that this specific prefetcher/workload combination should not be treated as bandwidth-representative of a tuned production system -- a caveat carried into Section 3.5, not resolved here.

3.5 Closing the Loop: Real Data in the Analytical Model

The real distributions measured in Sections 3.3-3.4 were fed into the calibrated analytical model's own functions (Section 2.4) in place of the swept gating_branches/alpha/total-overhead assumptions, to check whether the composed residual-bandwidth prediction survives contact with real data. Using the

real aggregate wrong-path rate (25.3%, weighted across the four trustworthy sites) and the real waste concentration (only 4 distinct prefetch-issuing sites were found in this workload, not the 32 originally assumed; the best-fit Zipf concentration parameter is $\alpha \approx 0.49$, mid-range within what was swept) -- combined with Magellan's literature-anchored overhead rather than this project's own under-representative measurement (Section 3.4) -- the composed residual comes out to 1.62% of one DRAM channel.

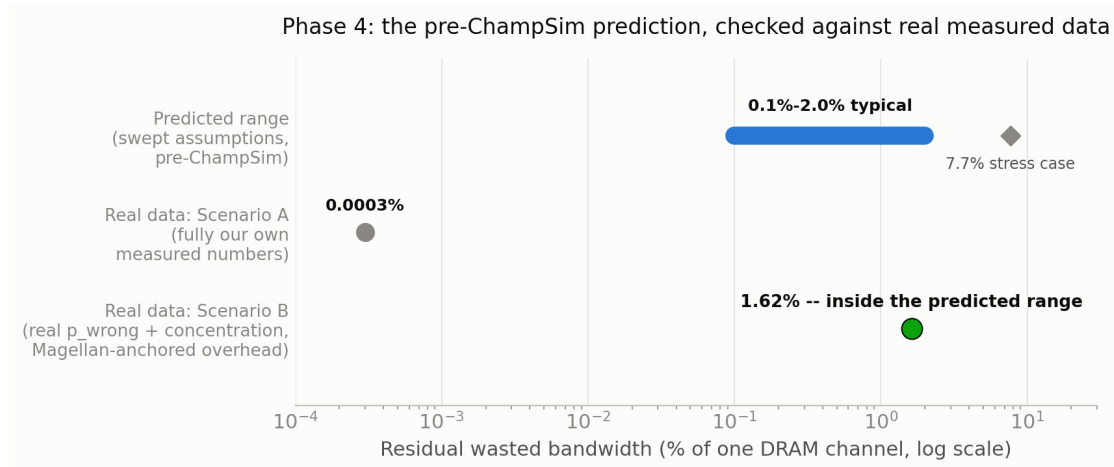


Figure 7. The composed model's prediction, made before any ChampSim data existed, against real measured data. The real-data result (green) lands inside the originally predicted typical range; using this project's own conservatively-tuned overhead measurement instead (gray) lands far below it, consistent with the Section 3.4 caveat rather than contradicting the model.

One discrepancy surfaced rather than resolved: the model's formula, given ChampSim's own real branch-accuracy statistics for this run (MPKI 2.456, 97.17% program-wide accuracy), predicts only 5.2% wrong-path at the real measured gating_branches (mean 1.843) -- about 5x below the 25.3% actually measured. Matching the real rate would require the specific identified gating branch to have a local accuracy near 85%, well below the program-wide average. This is consistent with that branch being a disproportionately hard-to-predict (H2P) one rather than an average branch, which a single global-accuracy figure cannot see -- and it directly reinforces this proposal's core premise that per-branch, not program-average, confidence is the signal worth gating prefetch dispatch on.

3.6 What This Does and Does Not Establish

This is real, corroborating empirical evidence for the analytical model's central claim, now closed into a full loop: a composed prediction made from swept literature-anchored assumptions alone (Section 2.4) was checked against real ChampSim measurements (Sections 3.3-3.4) and survived (Section 3.5) -- not merely

"plausible in principle" but landing inside the range predicted before any simulator time was spent. This is still not proof: the evidence comes from a single workload (mcf) over a short (5M-instruction) simulation window, one of five tracked load sites remains excluded as unmeasurable with the current method, and gating-branch identification remains a coarse IP-distance heuristic rather than genuine control-dependence analysis. The concrete next steps, in order, are: validate on 1-2 additional workloads and a longer window to check the 18.3%/1.62% figures hold up; investigate the H2P-branch discrepancy in Section 3.5 with direct per-branch accuracy instrumentation; and only then commit to building the mechanism proposed in Section 4.

4. Approach

4.1 Core Idea

Gate the dispatch of long-distance software prefetch instructions using a dynamic, per-branch confidence/reconvergence signal, so that prefetches issued deep into a low-confidence speculative window are delayed, downgraded, or suppressed before they reach the memory controller — rather than being throttled only after the fact, and rather than relying on aggregate, per-PC historical accuracy statistics that cannot see a single, high-cost, in-flight event.

4.2 Mechanism Sketch

1. Confidence signal generation: extend the branch predictor's existing confidence/reconvergence machinery (in the spirit of CHES's similarity-based reference-trace matching) to expose a per-branch, per-cycle confidence value for the current speculative path — i.e., how closely the in-flight instruction stream matches a known, high-confidence "golden path," and how many unresolved branches currently gate the path's correctness.
2. Prefetch tagging: at compile time, the software prefetcher (Magellan-style) tags each long-distance prefetch instruction with lightweight metadata identifying which branch(es) it is control-dependent on — i.e., the small set of "gating" branches whose resolution determines whether this specific prefetch's target will actually be used. This reuses the same dependence-graph analysis Magellan already performs to generate the prefetches, so it does not require new compiler infrastructure from scratch.
3. Dispatch-time gate: when a tagged prefetch reaches issue/dispatch, look up the current confidence signal for its gating branch(es). If confidence is above a

threshold, dispatch normally. If confidence is low, apply a graduated response rather than a binary block (see §4.3) — informed by the LLBP-X-derived (corrected: Mutlu et al. wrong-path usefulness) constraint that some wrong-path work is still beneficial, so a blanket “disable on divergence” policy is not appropriate.

4. Cost-aware differentiation: use estimated latency/cost of the prefetch target (e.g., likely LLC hit vs. likely DRAM-bound) to bias the gate — cheap, on-chip prefetches are allowed to proceed even under low confidence (bounded blast radius), while expensive, off-chip prefetches require higher confidence before dispatch (this is the class of request the Figures of Merit in §1.5 are built around).
5. Integration with existing reactive throttling: retain today's per-PC/per-stream accuracy counters for the concentrated, systematic waste they already catch well; layer the proactive, per-branch gate on top specifically to catch the diffuse, data-dependent (high-H2P) waste that reactive mechanisms structurally cannot see (§2.4, Q3).

4.3 Design Space for the Gate (Graduated, Not Binary)

- Delay: hold the prefetch in a small staging buffer for a few cycles until the gating branch resolves, rather than dispatching or dropping it immediately — recovers most of the latency-hiding benefit when the branch resolves quickly (most branches reconverge fast; see the analytical model's rationale for treating only a small number of branches as “gating” rather than every branch in the prefetch-distance window).
- Downgrade: convert a low-confidence long-distance DRAM-bound prefetch into a shorter-distance or LLC-only prefetch, reducing blast radius without fully discarding the request.
- Suppress: drop the prefetch entirely when confidence is very low and estimated cost is high — the only point in the design space equivalent to today's blanket throttling, reserved for the worst-case combination.
- Priority demotion at the memory controller: for prefetches that must be dispatched despite low confidence (e.g., already past the local cancellation window described in US Patent 6,438,656), demote their queue priority so they cannot block subsequent correct-path demand fetches — complementary to, not a replacement for, the dispatch-time gate.

5. Possible Solutions

5.1 Solution A — Static Confidence Threshold Gate (Low Complexity)

A single global confidence threshold, read from the existing branch predictor's confidence estimator, gates all tagged prefetches uniformly. Cheapest to implement; reuses predictor hardware already present. Weakness: a single global threshold cannot distinguish cheap from expensive prefetches, so it either under-gates (misses savings) or over-gates (loses legitimate latency-hiding benefit) depending on where the threshold is set. Best suited as a first-order baseline against which more nuanced schemes are measured.

5.2 Solution B — Per-Branch Reconvergence-Gated Dispatch (Primary Proposal)

The mechanism described in §4: per-branch dynamic confidence derived from a CHESS-style similarity/reference-trace predictor, compiler-tagged gating-branch metadata on each prefetch, and a graduated (delay/downgrade/suppress) dispatch policy rather than a binary gate. This directly targets the diffuse, high-H2P-density waste that reactive throttling cannot reach (§2.4, Q3), while the cost-aware tiering addresses the LLBP-X-corrected constraint that not all wrong-path work is worthless. Highest expected payoff; highest implementation complexity (requires new compiler metadata and a new dispatch-time hardware structure).

5.3 Solution C — Hybrid Reactive + Proactive Throttling

Keep today's per-PC/per-stream reactive accuracy counters as the first line of defense for concentrated, systematic waste, and layer the proactive per-branch gate (Solution B) on top only for prefetch sites where reactive counters report high aggregate accuracy despite the underlying branches being individually hard-to-predict — i.e., use the reactive signal itself to decide where the more expensive proactive mechanism is worth activating. Reduces the proactive mechanism's always-on hardware cost at the expense of a slightly more complex activation policy.

5.4 Solution D — Memory-Controller-Side Priority Demotion (Fallback / Complementary)

For prefetches that cannot be gated before the local cancellation window closes (per US Patent 6,438,656's early-cancellation boundary), demote their memory-

controller queue priority based on the same confidence signal, rather than gating dispatch itself. Lower implementation risk since it does not touch the front-end issue path, but only manages contention after the wasted request has already been made — addresses on-chip/queueing pollution (Figure of Merit §1.5) more than off-chip bandwidth waste, and should be paired with, not substituted for, Solution B.

5.5 Recommended Path

Solution B (per-branch reconvergence-gated dispatch) is the primary proposal, since it is the only option that directly targets the diffuse, per-event waste this idea's own novelty gap analysis identifies as unreachable by existing mechanisms. Solution A should be built first as a cheap baseline for comparison; Solution D should be included as a complementary mechanism for the unavoidable fraction of prefetches that cannot be gated in time. Before committing further gem5/ChampSim infrastructure time, the two concrete empirical tasks flagged by the validation audit should be resolved: (1) measure the actual average number of control-dependent gating branches per software prefetch in real graph/sparse workloads, and (2) measure how much of the resulting waste standard reactive throttling already recovers in practice, reconciling both against Magellan's own ~10% total-bandwidth-overhead ceiling.

6. References

- [1] Fu, G., Xia, T., Yin, M., Nair, P. J., Lis, M., & Ren, P. (2025). Magellan: A High-Performance Loop-Guided Prefetcher for Indirect Memory Access. In Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), Tokyo, Japan, pp. 601–615. <https://doi.org/10.1145/3695053.3731054>
- [2] Volos, H., Vassiliou, S., Antoniou, G., Bartolini, D. B., & Sazeides, Y. (2025). Leveraging control-flow similarity to reduce branch predictor cold effects in microservices. In Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), Tokyo, Japan. <https://doi.org/10.1145/3695053.3731059>
- [3] Schall, D., Sandberg, A., & Grot, B. (2024). The Last-Level Branch Predictor. In Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO '24).
- [4] Lee, C. J., Kim, H., Mutlu, O., & Patt, Y. N. (2008). Performance-Aware Speculation Control Using Wrong Path Usefulness Prediction. In Proceedings of the 14th International Symposium on High-Performance Computer Architecture (HPCA '08), Salt Lake City, UT, pp. 39–49.
- [5] U.S. Patent No. 6,438,656. Method and system for cancelling speculative cache prefetch requests.
- [6] General industry and patent literature on confidence/accuracy-based throttling for hardware (stream/next-line) prefetchers — cited as a class of adjacent, reactive prior art; no single representative patent independently verified for this document.
- [7] Gober, N., Chacon, G., Wang, L., Gratz, P. V., Jimenez, D. A., Teran, E., Pugsley, S., & Kim, J. (2022). The Championship Simulator: Architectural Simulation for Education and Competition. arXiv:2210.14324. (ChampSim, used for the empirical validation in Section 3.)